

Functional Dependency and Normalization

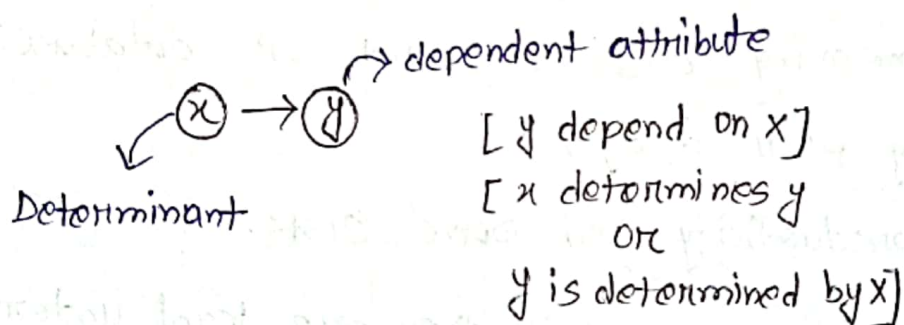
1. Dependency
2. Functional dependency, uses, Types, Rules
3. Closure of functional dependency
4. Normalization, Types (1NF - 5NF)
5. Anomaly
6. Decomposition

Dependency: A database dependency is a constraint that defines the relationship between attributes. It happens when information stored in the same database table uniquely and information stored in the same table.

Functional dependency: Functional dependency is a relationship ~~set~~ status between attributes. A functional dependency is a generalization of the notion of a key.

Functional dependency, $\alpha \rightarrow \beta$

If R is a relation with attributes X and Y , a functional dependency between the attributes is represented as $X \rightarrow Y$ which specifies that Y is functionally dependent on X . Here X is a determinant set and Y is a dependent attribute. Each value of X is associated with one Y value.



x	$y = 2x$
1	2
2	4

$x \rightarrow y$

Q Let R be a relation schema,

$\alpha \subseteq R$ and $\beta \subseteq R$

$$t_1[\alpha] = t_2[\alpha] \Rightarrow t_1[\beta] = t_2[\beta]$$

- ⓐ Functional dependencies allow us to express constraints that cannot be expressed using superkeys.

loan-info-schema = (customer-name, loan-number, branch-name, amount)

Hence, $\text{loan-number} \rightarrow \text{amount}$
 $\text{loan-number} \rightarrow \text{branch-name}$ } (✓)

$\text{loan-number} \rightarrow \text{customer-name}$ (X)

Uses of Functional dependency:

1. Prevent data Redundency
2. Maintain quality and integrity of data
3. Reduce the risk error
4. Define meaning and constraints of database
5. Identify poor design
6. Gain productivity and save costs.
7. Test relation to see if they are legal under a given set of functional dependencies.

→ যে attribute কোনো attribute দ্বারা মান বা বিন্যাসে determined হয়,

Closure of functional dependencies: The complete

set of all positive attributes that can be functionality derived from given functional using armstrongs rules.

Armstrongs rules: It provides a simple techniques from reasoning functional dependency.

(Dependency এর ক্ষেত্রে অন্য ৭ rules use করার ক্ষেত্রে বলা Armstrongs rules)

Closure of functional dependency,

→ set of attribute

→ derive from functional dependency

☐ $R(A, B, C, D)$

Functional dependency = $\{A \rightarrow B, B \rightarrow C, C \rightarrow D\} \rightarrow$ Candidate key

closure of A, $A^+ = \{A, B, C, D\}$

closure of B, $B^+ = \{B, C, D\}$

closure of C, $C^+ = \{C, D\}$

closure of D, $D^+ = \{D\}$

↓

[এ closure এ
এ attribute
আছে]

☐ Sound (generate only functional dependencies that actually hold)

Complete (generate all functional dependencies that hold)

Rules of functional dependency:

- Primary**
- ☐ Reflexivity $\Rightarrow$ If B is a subset of A then $A \rightarrow B$
 - ☐ Augmentation $\Rightarrow$ If $A \rightarrow B$ then $AC \rightarrow BC$
 - ☐ Transitivity $\Rightarrow$ If $A \rightarrow B$ and $B \rightarrow C$ then $A \rightarrow C$
- Secondary**
- ☐ Projectivity or Decomposition Rule $\Rightarrow$ If $A \rightarrow BC$ then $A \rightarrow B$ and $A \rightarrow C$
 - ☐ Union or Additive Rule $\Rightarrow$ If $A \rightarrow B$ and $A \rightarrow C$ then $A \rightarrow BC$
 - ☐ Pseudo Transitive Rule $\Rightarrow$ If $A \rightarrow B$, $DB \rightarrow C$ then $DA \rightarrow C$

Types of functional dependency:

- ① Trivial functional dependency
- ② Non Trivial functional dependency
- ③ Multivalued functional dependency
- ④ Partial functional dependency
- ⑤ Transitive functional dependency

Trivial functional dependency: A trivial dependency refers to a type of functional dependency between two attributes (columns) in a relation (table) that is self evident or obvious.

If $x \rightarrow y$ then y is a subset of x that is called trivial functional dependency.

ID, Name $\rightarrow$ Name

[Key $\rightarrow$ is a key dependency]

Non-trivial dependency: A non-trivial functional dependency is a concept related to the relationships between attributes in a database table.

$x \rightarrow y$ is called a Non-trivial functional dependency if y is not a subset of x . So, a functional dependency $x \rightarrow y$ where x is a set of attributes and y is also a set of the attributes but not subset of x , then it is called non-trivial functional dependency.

$sid \rightarrow Sname$
 $\downarrow \quad \quad \downarrow$
 key attribute Non key attribute

[key এর উপর non-key depend করলে]

Multivalued dependency: A multivalued functional dependency is a concept in database theory that extends the idea of functional dependencies.

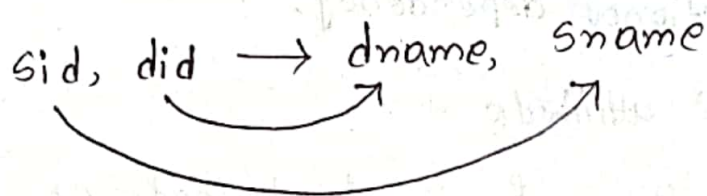
When two distinct attributes in a table are independent of each other but depend on a third attribute.

$sid \rightarrow Mobile, email$
 $\downarrow$
 multivalued

[একই attribute যদি Key এর উপর depend হয়]

Partial dependency: It is a concept in database and normalization that occurs when an attribute in a relational database depends on a part of a multivalued candidate key (primary key) rather than the entire candidate key.

The functional dependency $A \rightarrow B$ happens to be a partial dependency if B is functionally dependent on A and also B can be determined by any other proper subset of A .



[sid এর উপর dname
এবং did এর উপর
dname depend করে
তাই Partial, (মহাশয়) কেউ
মহাশয় হয় key এর অংশ
Part এর উপর non key
dependent]

Transitive dependency: Transitive dependency is a functional dependency which holds by virtue of transitivity. A transitive dependency can occur only in a relation that has 3 or more attributes.

$$A \rightarrow B$$

$$B \rightarrow C$$

$$A \rightarrow C$$

It is not the case that $B \rightarrow A$

Normalization: Normalization is the process of organizing the data in database.

Two basic requirements:

- ⇒ There is no redundancy of data
- ⇒ Data dependencies are logical

- ☐ Use to reduce data redundancy
- ☐ To improve data consistency
- ☐ Divides larger table into smaller table links them using relationship
- ☐ Form of decomposition
- ☐ Reduces anomalies (insert, update, delete)

Anomaly: Anomaly means kind of problem occurs in special case.

- ⇒ Insert Anomaly
- ⇒ Deletion Anomaly
- ⇒ Update Anomaly

Insert Anomaly: It is the inability to add data to the database due to the absence of other data. It refers to when one cannot insert a new tuple into a relationship due to lack of data.

Deletion Anomaly: A delete anomaly is the opposite of an insert anomaly. When a delete anomaly occurs it means that you cannot delete data from the table without having to delete the entire record. It refers to the situation where the deletion of data results in the unintended loss of some other important data.

Update Anomaly: An update Anomaly is a data consistency that results from data redundancy and a partial update.

update anomalies occur when updating data in a database, but the update affects multiple rows or columns unintentionally.

The update anomaly is when an update of a single data value requires multiple rows of data to be updated.

Types of normalization:

1. 1NF (1st Normal Form)
2. 2NF (Second Normal Form)
3. 3NF (Third normal form)
4. 4NF ~~4NF~~
5. 5NF

1NF (First Normal Form)

A relation schema is said to be in 1NF if it contains atomic value and have no repeating value.

⇒ R relational schema R is in 1NF if the domains of all attributes of R are atomic.

For a table to be 1NF it should follow the following rules:

1. It should contain only single (atomic) valued attributes / columns.
2. Values stored in a column should be of the

same domain.

3. All the columns in a table should have unique names.
4. The order in which data is stored doesn't matter.

Example: Consider a table which is not in 1NF.

Table: Student-Info

Stu_name	Age	Courses
A	20	Physics
B	21	Math, physics
C	20	physics

To transform it into 1NF we must separate data into multiple rows.

Student Info

Stu_name	Age	Courses
A	20	Phy
B	21	Math
B	21	Phy
C	20	Phy

2NF (Second Normal Form)

A relation schema is said to be 2NF if it is in 1NF and have no partial dependency. And all non-key attributes are fully functional dependent on the primary key.

For a table to be in 2NF,

1. It should be in 1NF
2. It should not have partial dependencies.

Table for 2NF would be

[Question
Answer table]

Example: Student-Info

Stu-name	Age
A	20
B	21
C	20

~~New table for 2NF would be,~~

Stu_name	Courses
A	Phy
B	Math
B	Phy
C	Phy

3NF (Third Normal Form)

A relation schema is said to be in 3NF if it is in 2NF and have no transitive dependency.

A table is said to be in 3NF when

⇒ It is in 2NF

⇒ It doesn't have any transitive dependency

Ex: Student-Info

Stu_id	Stu-name	DOB	Street	City	State	Zip
--------	----------	-----	--------	------	-------	-----

In this table stu-id is primary key but Street, City, and State depend on Zip. The dependency between Zip and other field

is called transitive dependency. Hence to apply 3NF, we need to move the street, city and state to new table with zip as primary key.

New 'stu_info' table

Stu-id	Stu-name	DOB	Zip
--------	----------	-----	-----

Newly introduced 'address' table

Zip	Street	City	State
-----	--------	------	-------

☐ Advantage of removing transitive dependency:

→ Amount of data duplication is reduced.

→ Data integrity is achieved.

☐ Ex: Banker-Schema = (branch-name, customer-name, banker-name)

banker-name → branch name

branch name Customer-name → banker-name

BENF (Boyce-Codd Normal Form)

BENF is a higher version of 3NF. This form deals with certain types of anomaly that is not handled by 3NF.

A 3NF table which doesn't have multiple overlapping candidate keys is said to be in BENF.

A relation schema is said to be BENF if it is in 3NF and there is no dependency in candidate key.

Condition: $\Rightarrow$ It must be in 3NF

$\Rightarrow$ For each functional dependency $X \rightarrow Y$, X should be a super key

Let us consider the following relationship: $R(A, B, C, D)$
and following dependencies:

$$A \rightarrow BCD$$

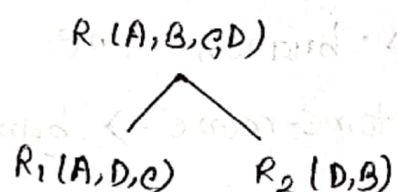
$$BC \rightarrow AD$$

$$D \rightarrow B$$

Above relationship is already in 3NF keys are A and BC .

Hence, in functional dependency, $A \rightarrow BCD$, A is a super key. In $BC \rightarrow AD$, BC is also a key. But in $D \rightarrow B$, D is not a key.

Hence, we break our relationship R into two Relationships, R_1 and R_2



Thus breaking into two tables one with A, D, C and the other with D and B .

4NF: A relation schema is said to be 4NF if it is in 3NF and BCNF and have no multivalued dependency.

Student

Ex:

Stu-id	course	Hobby
21	Computer	Dancing
21	Math	Singing
34	Chemistry	Dancing
74	Biology	Cricket
59	Physics	Hockey

In the given student table is in 3NF, but the course and hobby are two independent entity. There is no relationship between course and hobby.

Table for 4NF would be,

Stu-course

Stu-id	course
21	Computer
21	Math
34	Chemistry
74	Biology
59	Physics

Stu-Hobby

Stu-id	Hobby
21	Dancing
21	Singing
34	Dancing
74	Cricket
59	Hockey

In student relation a student with stu-id, 21 contains two courses, computer and Math and two hobbies, dancing and singing. So, there is a multivalued dependency on stu-id.

5NF: A relation schema is said to be 5NF and it contain 4NF and the decomposition will be lossless. Does not contain any dependency.

⇒ 5NF is satisfied when all the tables are broken into as many tables, as possible in order to avoid redundancy.

⇒ 5NF is also known as project-join normal form (PJ/NF)

Example:

subject	Lecturer	Semester
		1
Computer	Anshika	1
Computer	John	1
Math	John	2
Math	Akash	1
Chemistry	Praveen	

Table for 5NF would be,

P1

Semester	subject
1	Computer
1	Math
1	Chemistry
2	Math

P2

Subject	Lecturer
Computer	Anshika
Computer	John
Math	John
Math	Akash
Chemistry	Praveen

P3

Semester	Lecturer
1	Anshika
1	John
1	John
2	Akash
1	Praveen

Decomposition:

The process of decomposition in DBMS helps us remove redundancy, inconsistencies and anomalies from a database when we divide the table into numerous tables.

The process of decomposition refers to dividing relation

⇒ All attributes of an original schema (R) must appear in the decomposition (R_1, R_2):

$$R = R_1 \cup R_2$$

⇒ Lossless join decomposition

For all possible relations h on schema R

$$h = \pi_{R_1}(h) \bowtie \pi_{R_2}(h)$$

Example: Decomposition of $R = (A, B)$

$$R_1 = (A) \quad R_2 = (B)$$

A	B
α	1
α	2
β	1

h

A
α
β

$\pi_A(h)$

B
1
2

$\pi_B(h)$

$$\pi_A(h) \bowtie \pi_B(h)$$

A	B
α	1
α	2
β	1
β	2